PixelatedRF

Architecture Deep Dive

Forward Surrogate · Inverse MCL Transformer · Tandem Training

---

Phase A — Forward Surrogate

Layout (12×12) → CNN → S_{11} at 81 frequencies

Phase B — Inverse MCL Transformer

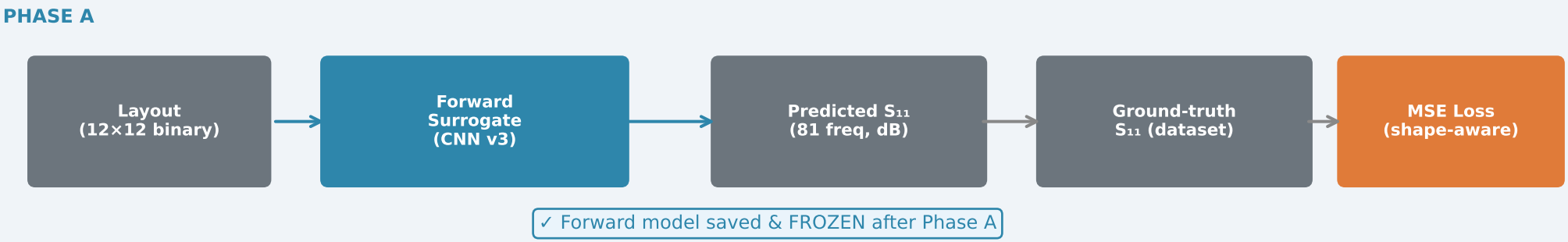
Target S_{11} → K parallel heads → best-of-K layout

Tandem Loss

Frozen Forward guides the Inverse during training

Pipeline Overview

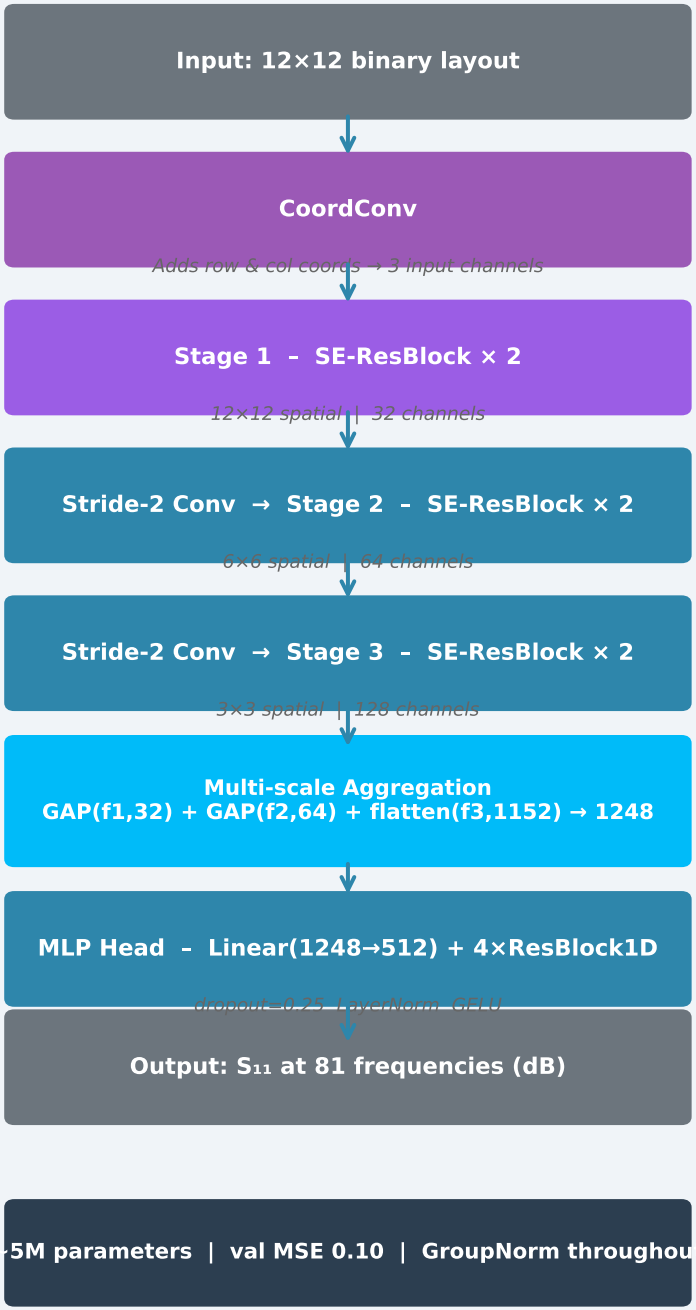
Two-phase tandem training — forward trained first, inverse trained second.



Key insight: the Inverse does NOT improve the Forward.
The Forward is used as a differentiable simulator to train the Inverse.

Forward Surrogate — Architecture v3

CoordConv + SE-ResNet + Multi-scale Aggregation → S_{11}



SE Block (Squeeze-and-Excitation)

After each ResBlock, learns WHICH channels matter for this specific layout.
E.g. "this layout has a 5 GHz resonance → amplify channels encoding that pattern."

CoordConv

Adds row/col coordinate channels [-1,+1].
A pixel at (0,0) vs (6,6) has completely different EM coupling to the feed point.
Plain CNN cannot distinguish position.

Multi-scale Aggregation

f1 (12×12) → global antenna topology
f2 (6×6) → mid-scale resonance context
f3 (3×3) → fine spatial resonance detail
All 3 concatenated: 1248-dim vector.

Shape-aware Loss

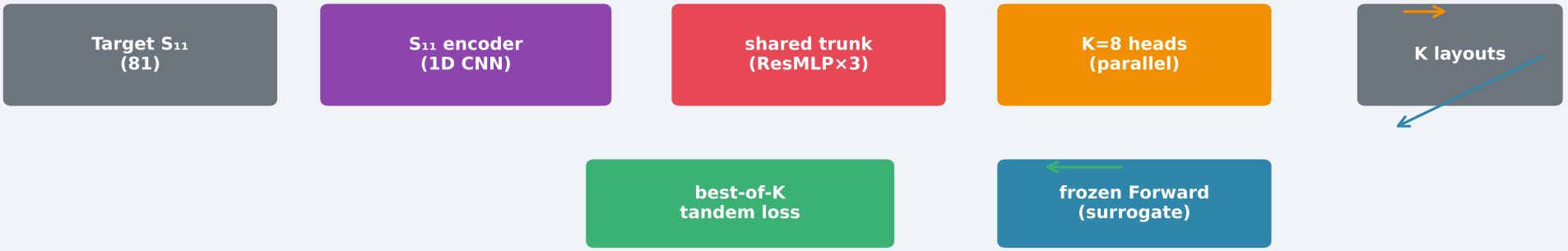
Deep dips get 3× weight (vs flat MSE).
Gradient matching enforces correct resonance edge slopes → sharper dips.
Result: val MSE dropped 0.23 → 0.10.

Inverse Design — Multi-Choice Learning (K=8 heads)

K parallel decoder heads solve the ONE-TO-MANY inverse problem (no latent z)

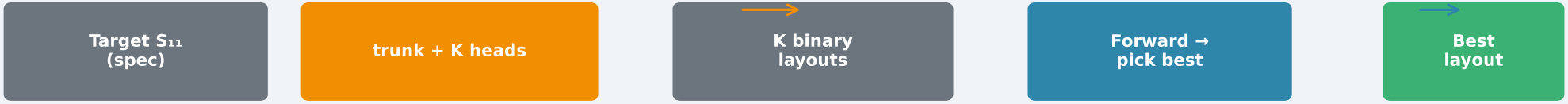
TRAINING (best-of-K tandem)

Target S_{11} → K candidate layouts → frozen Forward → only the BEST head is penalised (best-of-K), so each head specialises on a different solution.



INFERENCE (best-of-K)

One forward pass yields all K candidate layouts; pick the one whose surrogate S_{11} best matches the target. Deterministic — no sampling.



Why MCL (not a VAE / plain network)?

One-to-many problem

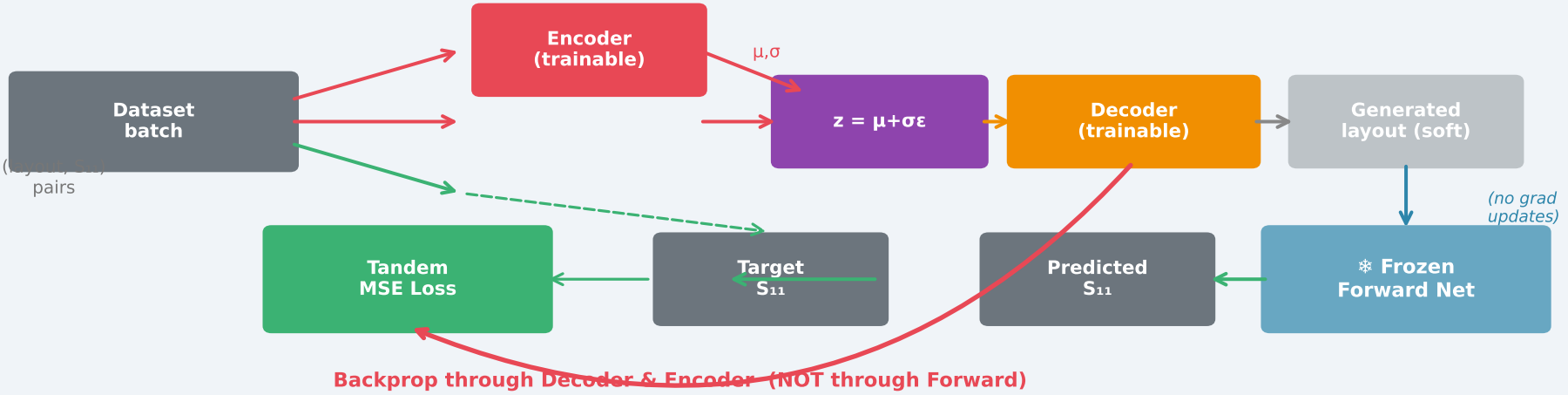
Many layouts produce the same S_{11} curve.
A deterministic net averages them → blurry grey pixels.
K competing heads each lock onto a distinct crisp solution.

Best-of-K = the metric

Loss penalises only the best head, exactly matching the best-of-K eval metric. No latent z, no posterior collapse, no KL tuning — just K specialised decoders.

Tandem Training — How the 3 Networks Interact

The fundamental problem: inverse design has no direct loss signal.
We cannot directly compute "how wrong" a layout is — we need to simulate it first.
The forward surrogate acts as a fast differentiable simulator for training the inverse.



Training Loss Breakdown (Phase B)

BCE × 0.1

Layout reconstruction
(weak — allows the decoder
to generate different valid
layouts, not just the gt one)

KL × β(cyclical)

Latent space regularization
(cyclical β: 0→0.5 per 20ep
prevents posterior collapse;
encoder stays engaged)

Shape-aware × 3.0

EM performance matching
(MAIN objective — upweights
deep resonance dips + enforces
correct resonance slopes)

Key Question: How Do the 3 Networks Relate?

⚠ **The Inverse does NOT improve the Forward. The Forward is FIXED after Phase A.**

Dependency & Data Flow

Does the Inverse make the Forward BETTER?

NO.

The Forward is FROZEN during Phase B.
Its weights never change.
Only the Inverse (encoder + decoder) receives gradient updates.

Does the Forward make the Inverse BETTER?

YES — critically.

The Inverse CANNOT be trained without the Forward. There is no direct loss on a layout — you need to simulate it first.
Better Forward → more accurate gradient signal → better Inverse.

Why not just skip the Inverse?

Speed.

Forward alone tells you S_{11} from layout.
Inverse lets you DESIGN: give a target S_{11}, get a layout instantly — without running thousands of expensive EM simulations.

Training order is strictly sequential: Phase A (Forward, ~2.5h) → Phase B (Inverse, ~1.5h) → Inference